Crypto Viz

Tableau de bord d'analyse technique en temps réel

Équipe de projet

— Adrien GARNIER
— Chi NGUYEN
— Nathan HAN
— Samir MIGUEL
— Zeyu HOU

Date : 11 décembre 2025

Sommaire

— **I / Introduction**
— **II / Architecture**
— **III / Les indicateurs**
  — RSI – Relative Strength Index
  — MACD – Moving Average Convergence Divergence
  — Bollinger Bands
  — Moving Averages (SMA/EMA)
  — Volume Analysis

1. Introduction

Crypto Viz est un tableau de bord d'analyse technique en temps réel dédié aux cryptomonnaies. Son objectif est de collecter en continu les données de marché, de les analyser automatiquement à l'aide d'indicateurs techniques reconnus, puis de les visualiser de manière dynamique afin d'aider les utilisateurs à identifier rapidement les opportunités et les risques.

Ce rapport présente d'abord l'architecture globale du système (flux de données, services de calcul, stockage et visualisation), puis détaille les différents indicateurs mis en œuvre, des métriques générales de marché aux indicateurs techniques avancés.

2. Architecture

L'architecture de Crypto Viz est organisée autour de trois composants principaux qui structurent le flux de données de bout en bout, depuis la collecte brute jusqu'à l'affichage temps réel des indicateurs.

— **Web Scrapper** : composant de collecte continue des données de marché (rôles de *producer*), chargé de récupérer en temps réel les prix, volumes et autres informations pertinentes depuis les API crypto.
— **Analytics Builder** : composant de traitement et d'analyse (rôles de *consumer* puis *producer*), qui ingère les données brutes, calcule les indicateurs techniques (RSI, MACD, Bandes de Bollinger, moyennes mobiles, volume, etc.) et génère des signaux exploitables.
— **Dynamic Viewer** : composant d'affichage en temps réel, qui consomme les résultats d'analyse et les présente sous forme de graphiques et de tableaux interactifs, avec une forte dimension temporelle (évolution des signaux dans le temps).

La figure suivante illustre schématiquement cette architecture et les échanges entre les trois composants :

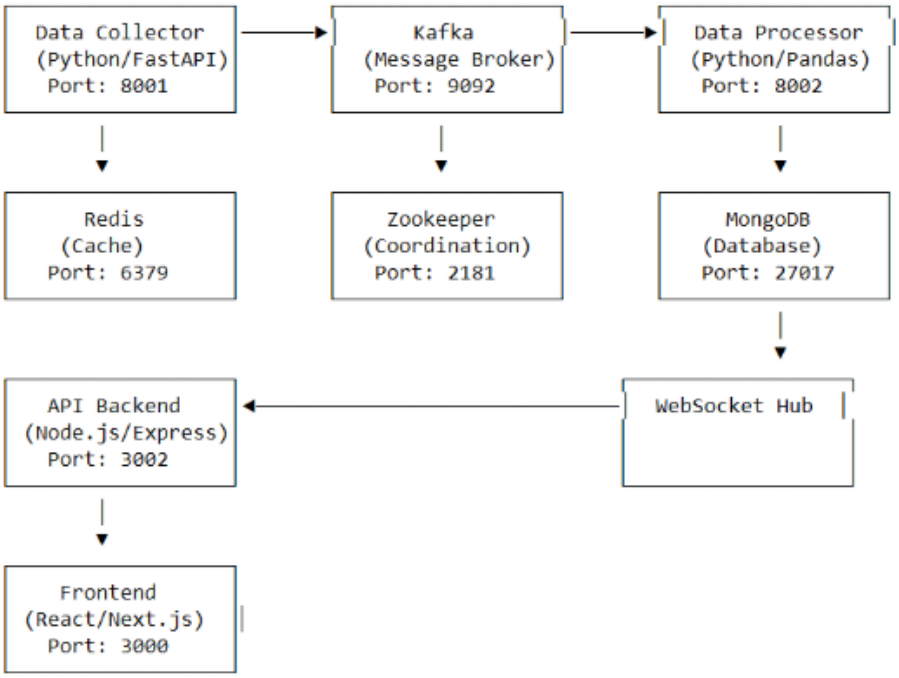


FIGURE 1 – Architecture globale de Crypto Viz : Web Scrapper, Analytics Builder et Dynamic Viewer.

3.1. RSI – Relative Strength Index

Définition

Le RSI (Relative Strength Index) est un indicateur de momentum qui mesure la vitesse et l'amplitude des mouvements de prix. Il oscille entre 0 et 100 et indique si un actif est suracheté (overbought) ou survendu (oversold).

Utilité et choix

Utilisations principales :
— Détection des zones de surachat (prix trop haut, correction possible).
— Détection des zones de survente (prix trop bas, rebond possible).
— Identification de divergences (prix monte mais RSI descend = faiblesse).
— Confirmation de tendances (RSI > 50 = haussier, RSI < 50 = baissier).
Pourquoi choisi pour ce projet :
— Simple à interpréter (un seul nombre entre 0 et 100).
— Très utilisé par les traders crypto.
— Excellent pour identifier les points d'entrée/sortie potentiels.
— Génère des signaux clairs pour les alertes automatiques.

Formules mathématiques

Étape 1 : Rendements de prix

$$\text{Rendement}(t) = \frac{\text{Prix}(t) - \text{Prix}(t-1)}{\text{Prix}(t-1)} = \frac{\text{Prix}(t)}{\text{Prix}(t-1)} - 1$$

$$\text{rendement_1h} = \frac{\text{prix_actuel} - \text{prix_1h_ago}}{\text{prix_1h_ago}} \times 100$$

$$\text{rendement_24h} = \frac{\text{prix_actuel} - \text{prix_24h_ago}}{\text{prix_24h_ago}} \times 100$$

$$\text{rendement_7j} = \frac{\text{prix_actuel} - \text{prix_7j_ago}}{\text{prix_7j_ago}} \times 100$$

Étape 2 : Gains et pertes

$$\text{Gain}_i = \begin{cases} \Delta_i & \text{si } \Delta_i > 0, \\ 0 & \text{sinon} \end{cases} \qquad \text{Perte}_i = \begin{cases} |\Delta_i| & \text{si } \Delta_i < 0, \\ 0 & \text{sinon} \end{cases}$$

Étape 3 : Moyennes sur $n = 14$ périodes

$$\text{MoyenneGains} = \frac{\sum \text{Gain}_i}{n}, \qquad \text{MoyennePertes} = \frac{\sum \text{Perte}_i}{n}$$

Étape 4 : Relative Strength

$$RS = \frac{\text{MoyenneGains}}{\text{MoyennePertes}}$$

Étape 5 : RSI

$$RSI = 100 - \frac{100}{1 + RS}$$

Forme simplifiée :

$$RSI = 100 \times \frac{\text{MoyenneGains}}{\text{MoyenneGains} + \text{MoyennePertes}}$$

Implémentation (exemple de code)

Listing 1 – RSI computation

```
def _calculate_rsi(self, df: pd.DataFrame, period: int = 14):
    """
    Computes the RSI (Relative Strength Index)

    Args:
        df: DataFrame with 'close' column
        period: Lookback period (default: 14)

    Returns:
        RSIIndicator with value and signals
    """
    # Step 1: Compute price changes
    delta = df['close'].diff()

    # Step 2: Separate gains and losses
    gain = delta.where(delta > 0, 0)  # Keep only gains
    loss = -delta.where(delta < 0, 0) # Keep only losses (as positive
       values)

    # Step 3: Moving averages of gains and losses
    avg_gain = gain.rolling(window=period).mean()
    avg_loss = loss.rolling(window=period).mean()

    # Step 4: Compute RS
```

```
    rs = avg_gain / avg_loss

    # Step 5: Compute RSI
    rsi = 100 - (100 / (1 + rs))

    current_rsi = float(rsi.iloc[-1])

    return RSIIndicator(
        value=current_rsi,
        overbought=current_rsi > 70,  # Overbought signal
        oversold=current_rsi < 30,    # Oversold signal
        timestamp=datetime.utcnow()
    )
```

Interprétation du RSI

Valeur RSI	Interprétation	Action suggérée
RSI > 70	Surachat (Overbought)	Prudence, correction possible, signal de vente potentiel.
$50 < RSI < 70$	Tendance haussière saine	Position haussière viable.
RSI $= 50$	Neutre / Équilibre	Pas de signal clair.
$30 < RSI < 50$	Tendance baissière	Position baissière ou attente.
RSI < 30	Survente (Oversold)	Opportunité de rebond, signal d'achat potentiel.

Alertes RSI (exemple)

Listing 2 – Alertes RSI

```
# Alerte RSI Overbought
if indicators.rsi.overbought:  # RSI > 70
    alert = Alert(
        symbol="BTC/USDT",
        alert_type="rsi_overbought",
        severity="MEDIUM",
        message="BTC/USDT RSI is overbought at 72.5",
        value=72.5,
        threshold=70.0
    )

# Alerte RSI Oversold
if indicators.rsi.oversold:  # RSI < 30
    alert = Alert(
        symbol="ETH/USDT",
        alert_type="rsi_oversold",
        severity="MEDIUM",
        message="ETH/USDT RSI is oversold at 28.3",
        value=28.3,
        threshold=30.0
    )
```

Exemple concret RSI

Bitcoin (BTC/USDT) :

— Prix actuel : 67 650 $
— RSI(14) : 72.8
Interprétation :
— RSI > 70 : zone de surachat.
— Le prix a augmenté trop rapidement.
— Risque de correction baissière à court terme.
— Les traders peuvent prendre des profits.
— Alerte : `"BTC/USDT RSI is overbought at 72.8"`.

3.2. MACD – Moving Average Convergence Divergence

Définition

Le MACD est un indicateur de momentum qui montre la relation entre deux moyennes mobiles exponentielles (EMA). Il se compose d'une ligne MACD, d'une ligne signal et d'un histogramme.

Composants

— Ligne MACD : différence entre EMA rapide et EMA lente.
— Ligne Signal : EMA de la ligne MACD.
— Histogramme : différence entre MACD et Signal.

Utilité et choix

Utilisations principales :
— Détection des crossovers (signaux d'achat/vente).
— Identification du momentum (force de la tendance).
— Confirmation des changements de tendance.
— Détection de divergences prix / indicateur.
Pourquoi choisi pour ce projet :
— Génère des signaux précis et exploitables (crossovers).
— Combine tendance et momentum.
— Très populaire dans le trading crypto.
— Excellent pour des alertes automatiques basées sur événements.

Paramètres standards

— EMA rapide (*Fast*) : 12 périodes.
— EMA lente (*Slow*) : 26 périodes.
— Signal : 9 périodes.

Formules mathématiques

EMA générale

$$EMA_t = \text{Prix}_t \times \alpha + EMA_{t-1} \times (1 - \alpha)$$

avec

$$\alpha = \frac{2}{n+1}$$

EMAs pour le MACD

$$EMA_{\text{fast}} = EMA(\text{close}, 12), \qquad EMA_{\text{slow}} = EMA(\text{close}, 26)$$

Ligne MACD

$$MACD = EMA_{\text{fast}} - EMA_{\text{slow}}$$

Ligne Signal

$$\text{Signal} = EMA(MACD, 9)$$

Histogramme

$$\text{Histogramme} = MACD - \text{Signal}$$

Implémentation (exemple de code)

Listing 3 – MACD computation

```
def _calculate_macd(self, df: pd.DataFrame,
                    fast: int = 12, slow: int = 26, signal: int = 9):
    """
    Computes the MACD (Moving Average Convergence Divergence)

    Args:
        df: DataFrame with 'close' column
        fast: Fast EMA period (12)
        slow: Slow EMA period (26)
        signal: Signal line period (9)

    Returns:
        MACDIndicator with values and crossover signals
    """
    # Step 1: Compute EMAs
    ema_fast = df['close'].ewm(span=fast, adjust=False).mean()
    ema_slow = df['close'].ewm(span=slow, adjust=False).mean()

    # Step 2: MACD line
    macd_line = ema_fast - ema_slow

    # Step 3: Signal line (EMA of MACD)
    signal_line = macd_line.ewm(span=signal, adjust=False).mean()

    # Step 4: Histogram
    histogram = macd_line - signal_line

    # Detect crossovers (sign change of the histogram)
    prev_histogram = float(histogram.iloc[-2])
    curr_histogram = float(histogram.iloc[-1])

    # Bullish crossover: histogram goes from negative to positive
    bullish_crossover = prev_histogram < 0 and curr_histogram > 0

    # Bearish crossover: histogram goes from positive to negative
    bearish_crossover = prev_histogram > 0 and curr_histogram < 0

    return MACDIndicator(
        macd=float(macd_line.iloc[-1]),
        signal=float(signal_line.iloc[-1]),
        histogram=curr_histogram,
        bullish_crossover=bullish_crossover,
        bearish_crossover=bearish_crossover,
        timestamp=datetime.utcnow()
```

```
    )
```

Interprétation du MACD

Signal 1 : Crossovers

Événement	Condition	Action
Bullish Crossover	MACD croise Signal vers le haut	Achat, momentum haussier.
Bearish Crossover	MACD croise Signal vers le bas	Vente, momentum baissier.

Signal 2 : Position des lignes

— $MACD >$ Signal : momentum haussier.
— $MACD <$ Signal : momentum baissier.
— Histogramme > 0 : force acheteuse.
— Histogramme < 0 : force vendeuse.

Signal 3 : Amplitude de l'histogramme

— |Histogramme| élevé → momentum fort.
— |Histogramme| faible → momentum faible / consolidation.

Alertes MACD (exemple)

Listing 4 – MACD alerts

```
# MACD Bullish Crossover alert
if indicators.macd.bullish_crossover:
    alert = Alert(
        symbol="BTC/USDT",
        alert_type="macd_bullish_crossover",
        severity="HIGH",  # High importance!
        message="BTC/USDT MACD bullish crossover detected",
        value=5.3,  # Positive histogram
        threshold=0.0
    )
    # Strong BUY signal

# MACD Bearish Crossover alert
if indicators.macd.bearish_crossover:
    alert = Alert(
        symbol="ETH/USDT",
        alert_type="macd_bearish_crossover",
        severity="HIGH",
        message="ETH/USDT MACD bearish crossover detected",
        value=-4.7,  # Negative histogram
        threshold=0.0
    )
    # Strong SELL signal
```

Exemple concret MACD

Ethereum (ETH/USDT) :
— Prix : 3 420 $
— MACD : 15.2

— Signal : 12.8
— Histogramme : +2.4
Analyse :
— $MACD >$ Signal : momentum haussier.
— Histogramme positif : force acheteuse.
— Si l'histogramme était précédemment négatif, bullish crossover détecté.
— Action : signal d'achat, momentum haussier confirmé.

3.3. Bollinger Bands

Définition

Les Bandes de Bollinger sont un indicateur de volatilité composé de trois bandes : supérieure, médiane et inférieure.

Composants

— Bande supérieure (Upper) : moyenne + 2 écarts-types.
— Bande médiane (Middle) : moyenne mobile simple (SMA 20).
— Bande inférieure (Lower) : moyenne − 2 écarts-types.

Utilité et choix

Utilisations principales :
— Mesure de la volatilité du marché.
— Détection des prix extrêmes (surachat/survente).
— Identification des breakouts (cassures de bandes).
— Zones de support/résistance dynamiques.
Pourquoi choisi pour ce projet :
— Visualise clairement les zones de prix anormaux.
— Bandes élargies : haute volatilité.
— Bandes resserrées : consolidation, souvent avant un mouvement important.
— Breakouts génèrent des signaux puissants.

Paramètres standards

— Période $n = 20$.
— Multiplicateur d'écart-type $k = 2$.

Formules mathématiques

Bande médiane (SMA)

$$\text{Middle} = SMA(\text{close}, 20) = \frac{\sum \text{Prix}_i}{20}$$

Écart-type

$$\sigma = \sqrt{\frac{\sum(\text{Prix}_i - \text{Middle})^2}{20}}$$

Bandes supérieure et inférieure

$$\text{Upper} = \text{Middle} + k\sigma, \qquad \text{Lower} = \text{Middle} - k\sigma$$

Indicateur %B

$$\%B = \frac{\text{Prix actuel} - \text{Lower}}{\text{Upper} - \text{Lower}}$$

Interprétation :
— $\%B > 1$: prix au-dessus de la bande supérieure.
— $0 < \%B < 1$: prix entre les bandes.
— $\%B < 0$: prix en-dessous de la bande inférieure.

Implémentation (exemple de code)

Listing 5 – Bollinger Bands

```
def _calculate_bollinger_bands(self, df: pd.DataFrame,
                               period: int = 20, std_dev: int = 2):
    """
    Computes Bollinger Bands

    Args:
        df: DataFrame with 'close' column
        period: SMA period (20)
        std_dev: Number of standard deviations (2)

    Returns:
        BollingerBandsIndicator with bands and breakout signals
    """
    # Step 1: Middle band (SMA)
    middle = df['close'].rolling(window=period).mean()

    # Step 2: Standard deviation
    std = df['close'].rolling(window=period).std()

    # Step 3: Upper and lower bands
    upper = middle + (std_dev * std)
    lower = middle - (std_dev * std)

    # Current price
    current_price = float(df['close'].iloc[-1])
    upper_band = float(upper.iloc[-1])
    lower_band = float(lower.iloc[-1])
    middle_band = float(middle.iloc[-1])

    # %B (position within the bands)
    percent_b = (current_price - lower_band) / (upper_band - lower_band)

    # Bandwidth (volatility)
    bandwidth = (upper_band - lower_band) / middle_band

    return BollingerBandsIndicator(
        upper=upper_band,
        middle=middle_band,
        lower=lower_band,
        percent_b=percent_b,
        bandwidth=bandwidth,
        breakout_upper=current_price > upper_band,  # Price > upper band
        breakout_lower=current_price < lower_band,  # Price < lower band
        timestamp=datetime.utcnow()
    )
```

Interprétation des Bandes de Bollinger

Signal 1 : Breakouts

Événement	Condition	Interprétation
Upper Breakout	Prix > Upper	Surachat, momentum très fort.
Lower Breakout	Prix < Lower	Survente, rebond possible.
Dans les bandes	Lower < Prix < Upper	Situation normale.

Signal 2 : Position %B

— $\%B > 1.0$: au-dessus de Upper, surachat extrême.
— $0.8 < \%B < 1.0$: proche de Upper, zone de surachat.
— $0.5 < \%B < 0.8$: partie haute, tendance haussière saine.
— $0.2 < \%B < 0.5$: partie basse, tendance baissière.
— $0 < \%B < 0.2$: près de Lower, zone de survente.
— $\%B < 0$: en-dessous de Lower, survente extrême.

Signal 3 : Largeur des bandes

— Bandwidth élevé : forte volatilité, mouvement en cours.
— Bandwidth faible : faible volatilité, consolidation, possible breakout imminent.

Alerte Bollinger (exemple)

Listing 6 – Alerte Bollinger

```
# Alerte Bollinger Breakout
if indicators.bollinger_bands.breakout_upper or indicators.
    bollinger_bands.breakout_lower:
     direction = "upper" if indicators.bollinger_bands.breakout_upper
         else "lower"

     alert = Alert(
          symbol="SOL/USDT",
          alert_type="bollinger_breakout",
          severity="HIGH",
          message=f"SOL/USDT Bollinger Bands {direction} breakout detected
              ",
          value=indicators.bollinger_bands.percent_b,
          threshold=1.0 if breakout_upper else 0.0
     )
     # Extreme volatility, strong movement in progress
```

Exemple concret Bollinger

Solana (SOL/USDT) :
— Prix actuel : 152.30 $
— Upper : 148.50 $
— Middle : 145.00 $
— Lower : 141.50 $
— %B : 1.54
Analyse :
— Prix > Upper : upper breakout.
— $\%B = 1.54$: prix largement au-dessus de la bande supérieure.

— Bandwidth élevé : forte volatilité.
— Momentum haussier très fort mais risque de correction.
— Alerte de sévérité élevée générée.

3.4. Moving Averages (SMA/EMA)

Définition

Les moyennes mobiles lissent les variations de prix pour identifier la tendance générale. Les deux types principaux sont la SMA et l'EMA.

Types de moyennes

— SMA (Simple Moving Average) : moyenne arithmétique simple.
— EMA (Exponential Moving Average) : moyenne pondérée qui donne plus de poids aux prix récents.

Utilité et choix

Utilisations principales :
— Identification de la tendance (haussière, baissière, neutre).
— Zones de support/résistance dynamiques.
— Golden Cross (signal haussier fort).
— Death Cross (signal baissier fort).
— Filtrage du bruit du marché.
Pourquoi choisi pour ce projet :
— Indicateur de base en analyse technique.
— Facile à interpréter.
— Périodes multiples pour court, moyen et long terme.
— Crossovers génèrent des signaux puissants.

Formules SMA et EMA

SMA

$$SMA_n = \frac{\sum_{i=1}^{n} \text{Prix}_i}{n}$$

Exemple :

$$SMA(20) = \frac{P_1 + P_2 + \cdots + P_{20}}{20}$$

EMA

$$EMA_t = \text{Prix}_t \times \alpha + EMA_{t-1} \times (1 - \alpha)$$

avec

$$\alpha = \frac{2}{n+1}$$

Exemples de facteur de lissage :
— EMA(12) : $\alpha = \frac{2}{13} \approx 0.1538$.
— EMA(26) : $\alpha = \frac{2}{27} \approx 0.0741$.

Périodes utilisées

— Court terme : EMA(12), EMA(26), SMA(20).
— Moyen terme : SMA(50).
— Long terme : SMA(200).

Implémentation (exemple de code)

Listing 7 – Moyennes mobiles

```
def _calculate_moving_averages(self, df: pd.DataFrame):
    """
    Calcule les moyennes mobiles (SMA et EMA)

    Args:
        df: DataFrame avec colonne 'close'

    Returns:
        MovingAverages avec toutes les moyennes
    """
    # Simple Moving Averages (SMA)
    sma_20 = df['close'].rolling(window=20).mean()
    sma_50 = df['close'].rolling(window=50).mean()
    sma_200 = df['close'].rolling(window=200).mean()

    # Exponential Moving Averages (EMA)
    ema_12 = df['close'].ewm(span=12, adjust=False).mean()
    ema_26 = df['close'].ewm(span=26, adjust=False).mean()

    # Prix actuel
    current_price = float(df['close'].iloc[-1])

    # Extraction des valeurs actuelles
    return MovingAverages(
        sma_20=float(sma_20.iloc[-1]) if len(sma_20) >= 20 else None,
        sma_50=float(sma_50.iloc[-1]) if len(sma_50) >= 50 else None,
        sma_200=float(sma_200.iloc[-1]) if len(sma_200) >= 200 else None
            ,
        ema_12=float(ema_12.iloc[-1]),
        ema_26=float(ema_26.iloc[-1]),
        current_price=current_price,
        timestamp=datetime.utcnow()
    )
```

Interprétation des moyennes mobiles

Signal 1 : Position du prix

- Prix > SMA(200) : très haussier, tendance long terme positive.
- Prix > SMA(50) : haussier, tendance moyen terme positive.
- Prix > SMA(20) : haussier court terme.
- Prix < SMA(20), SMA(50) ou SMA(200) : signaux baissiers correspondants.

Signal 2 : Ordre des moyennes

- Tendance haussière forte :

$$\text{Prix} > EMA(12) > EMA(26) > SMA(20) > SMA(50) > SMA(200)$$

- Tendance baissière forte :

$$\text{Prix} < EMA(12) < EMA(26) < SMA(20) < SMA(50) < SMA(200)$$

Signal 3 : Golden Cross / Death Cross

— Golden Cross : SMA(50) croise SMA(200) vers le haut (signal d'achat).
— Death Cross : SMA(50) croise SMA(200) vers le bas (signal de vente).

Signal 4 : Supports et résistances dynamiques

— Tendance haussière : SMA(20) et SMA(50) servent de supports.
— Tendance baissière : SMA(20) et SMA(50) servent de résistances.

Analyse automatique de tendance (exemple de code)

Listing 8 – Analyse de tendance

```
def analyze_trend(indicators: TechnicalIndicators) -> Tuple[str, float]:
    """Analyse la tendance base sur les moyennes mobiles"""

    bullish_signals = 0
    bearish_signals = 0

    # Signal 1 : Comparaison EMA court terme
    if indicators.moving_averages.ema_12 > indicators.moving_averages.
       ema_26:
        bullish_signals += 1  # Golden Cross court terme
    else:
        bearish_signals += 1  # Death Cross court terme

    # Signal 2 : Position du prix vs SMA(20)
    if indicators.moving_averages.current_price > indicators.
       moving_averages.sma_20:
        bullish_signals += 1
    else:
        bearish_signals += 1

    # Signal 3 : Position du prix vs SMA(50)
    if indicators.moving_averages.current_price > indicators.
       moving_averages.sma_50:
        bullish_signals += 1
    else:
        bearish_signals += 1

    # Signal 4 : Position du prix vs SMA(200) (poids double)
    if indicators.moving_averages.current_price > indicators.
       moving_averages.sma_200:
        bullish_signals += 2  # Plus important
    else:
        bearish_signals += 2

    # Calcul tendance et confiance
    total = bullish_signals + bearish_signals
    if bullish_signals > bearish_signals:
        return "bullish", bullish_signals / total
    elif bearish_signals > bullish_signals:
        return "bearish", bearish_signals / total
    else:
        return "neutral", 0.5
```

Exemple concret moyennes mobiles

Bitcoin (BTC/USDT) :
— Prix actuel : 67 650 $
— EMA(12) : 67 200 $
— EMA(26) : 66 800 $
— SMA(20) : 67 000 $
— SMA(50) : 65 500 $
— SMA(200) : 62 000 $
Analyse :
— Prix supérieur à toutes les moyennes : tendance très haussière.
— EMA(12) > EMA(26) : golden cross court terme.
— Prix > SMA(200) : tendance long terme positive forte.
— SMA(50) > SMA(200) : golden cross moyen terme actif.
Conclusion : tendance haussière confirmée sur tous les horizons, confiance élevée.

3.5. Volume Analysis

Définition

L'analyse de volume mesure le nombre d'unités échangées sur une période. Un mouvement de prix accompagné de fort volume est plus significatif qu'avec un volume faible.

Utilité et choix

Utilisations principales :
— Confirmation des tendances (volume élevé = tendance forte).
— Détection de spikes (intérêt inhabituel du marché).
— Identification d'anomalies (news, événements majeurs).
— Validation des breakouts (cassure + volume).
Pourquoi choisi pour ce projet :
— Complète les indicateurs basés sur le prix.
— Volume anormal souvent précurseur de mouvements importants.
— Facile à détecter par comparaison avec la moyenne.

Métriques de volume

Volume moyen

$$\text{VolumeAverage}_n = \frac{\sum \text{Volume}_i}{n}$$

Volume Ratio

$$\text{VolumeRatio} = \frac{\text{VolumeCurrent}}{\text{VolumeAverage}}$$

Interprétation :
— Ratio > 2.0 : spike de volume significatif.
— Ratio > 3.0 : spike extrême.
— Ratio < 0.5 : volume très faible.

Volume Oscillator

$$\text{VolumeOscillator} = \frac{\text{VolumeFast} - \text{VolumeSlow}}{\text{VolumeSlow}} \times 100$$

On-Balance Volume (OBV)

$$OBV_t = \begin{cases} OBV_{t-1} + \text{Volume}_t & \text{si Prix}_t > \text{Prix}_{t-1}, \\ OBV_{t-1} - \text{Volume}_t & \text{si Prix}_t < \text{Prix}_{t-1}, \\ OBV_{t-1} & \text{sinon} \end{cases}$$

Implémentation (exemple de code)

Listing 9 – Volume indicators

```
def _calculate_volume_indicators(self, df: pd.DataFrame, period: int =
    20):
    """
    Computes volume indicators

    Args:
        df: DataFrame with 'volume' column
        period: Period for the average (20)

    Returns:
        VolumeIndicators with volume metrics
    """
    # Current volume
    current_volume = float(df['volume'].iloc[-1])

    # Average volume over n periods
    avg_volume = df['volume'].rolling(window=period).mean()
    average_volume = float(avg_volume.iloc[-1])

    # Current volume / average volume ratio
    volume_ratio = current_volume / average_volume if average_volume > 0
        else 1.0

    # Fast/slow volume for oscillator
    volume_fast = df['volume'].rolling(window=5).mean()
    volume_slow = df['volume'].rolling(window=20).mean()
    volume_oscillator = ((volume_fast.iloc[-1] - volume_slow.iloc[-1]) /
        volume_slow.iloc[-1]) * 100

    # On-Balance Volume (OBV)
    obv = calculate_obv(df)

    # Spike detection (threshold: 2x the average)
    volume_spike = volume_ratio > 2.0

    return VolumeIndicators(
        current_volume=current_volume,
        average_volume=average_volume,
        volume_ratio=volume_ratio,
        volume_oscillator=float(volume_oscillator),
        obv=float(obv.iloc[-1]),
        volume_spike=volume_spike,
        timestamp=datetime.utcnow()
    )

def calculate_obv(df: pd.DataFrame) -> pd.Series:
    """Computes On-Balance Volume (OBV)"""
```

```
    obv = [0]
    for i in range(1, len(df)):
        if df['close'].iloc[i] > df['close'].iloc[i-1]:
            obv.append(obv[-1] + df['volume'].iloc[i])
        elif df['close'].iloc[i] < df['close'].iloc[i-1]:
            obv.append(obv[-1] - df['volume'].iloc[i])
        else:
            obv.append(obv[-1])
    return pd.Series(obv, index=df.index)
```

Interprétation des volumes

Signal 1 : Volume Ratio

Volume Ratio	Interprétation	Action
> 3.0	Spike extrême	Vérifier news, forte volatilité.
2.0–3.0	Spike significatif	Intérêt inhabituel, surveillance accrue.
1.5–2.0	Volume élevé	Mouvement confirmé, tendance forte.
0.8–1.5	Volume normal	Activité habituelle.
< 0.8	Volume faible	Faible conviction, mouvements peu fiables.

Signal 2 : Volume et direction du prix

— Prix en hausse + volume élevé : signal très haussier.
— Prix en hausse + volume faible : haussier faible, possible correction.
— Prix en baisse + volume élevé : signal très baissier.
— Prix en baisse + volume faible : baissier faible, rebond possible.

Signal 3 : OBV

— OBV monte avec le prix : tendance haussière confirmée.
— OBV baisse avec le prix : tendance baissière confirmée.
— OBV monte alors que le prix baisse : accumulation cachée, rebond possible.
— OBV baisse alors que le prix monte : distribution cachée, correction possible.

Alerte Volume (exemple)

Listing 10 – Volume alert

```
# Volume Spike alert
if indicators.volume_indicators.volume_spike:  # Ratio > 2.0
    alert = Alert(
        symbol="BNB/USDT",
        alert_type="volume_spike",
        severity="MEDIUM",
        message=f"BNB/USDT volume spike detected (ratio: {volume_ratio
            :.2f}x)",
        value=volume_ratio,
        threshold=2.0
    )
    # Unusual interest, check news/events
```

Exemple concret volume

Cardano (ADA/USDT) :
— Volume actuel : 45 230 000 ADA.
— Volume moyen (20) : 15 400 000 ADA.
— Volume Ratio : 2.94x.
— Prix : +8.5% (hausse).
Analyse :
— Volume Ratio proche de 3x : spike significatif.
— Prix en forte hausse avec volume élevé : signal très haussier confirmé.
— OBV en hausse : accumulation.

Synthèse : Pourquoi ces 5 indicateurs ?

Complémentarité

Indicateur	Mesure	Information	Utilisation
RSI	Momentum	Force acheteurs/vendeurs	Zones extrêmes.
MACD	Momentum + tendance	Changements de momentum	Signaux entrée/sortie.
Bollinger	Volatilité + prix	Écart à la normalité	Breakouts, extrêmes.
Moyennes	Tendance	Direction du marché	Confirmation LT.
Volume	Conviction	Force du mouvement	Validation signaux prix.

Stratégie multi-indicateurs

Exemple de signal d'achat fort pour BTC/USDT :
— RSI : 35, survente, rebond possible.
— MACD : bullish crossover détecté.
— Bollinger : prix touche la bande inférieure.
— Moyennes mobiles : prix au-dessus de SMA(200), tendance long terme haussière.
— Volume : spike de volume (2.8x), fort intérêt acheteur.
Les cinq indicateurs convergent alors vers un signal d'achat très fort avec confiance élevée.

Tableau de décision automatisé (exemple de code)

Listing 11 – Trading signal evaluation

```
def evaluate_trading_signal(indicators: TechnicalIndicators) -> Dict:
    """Evaluates the strength of a trading signal"""

    score_bullish = 0
    score_bearish = 0

    # RSI
    if indicators.rsi.oversold:
        score_bullish += 2
    if indicators.rsi.overbought:
        score_bearish += 2

    # MACD
    if indicators.macd.bullish_crossover:
        score_bullish += 3
    if indicators.macd.bearish_crossover:
```

```
            score_bearish += 3

        # Bollinger Bands
        if indicators.bollinger_bands.breakout_lower:
            score_bullish += 2
        if indicators.bollinger_bands.breakout_upper:
            score_bearish += 2

        # Moving Averages
        if indicators.moving_averages.ema_12 > indicators.moving_averages.
           ema_26:
            score_bullish += 1
        else:
            score_bearish += 1

        # Volume (multiplier)
        if indicators.volume_indicators.volume_spike:
            score_bullish *= 1.5
            score_bearish *= 1.5

        # Final decision
        if score_bullish > score_bearish * 1.5:
            return {"signal": "BUY", "strength": "STRONG", "score":
               score_bullish}
        elif score_bearish > score_bullish * 1.5:
            return {"signal": "SELL", "strength": "STRONG", "score":
               score_bearish}
        else:
            return {"signal": "HOLD", "strength": "NEUTRAL", "score": 0}
```